MAI 600 - Module 7 Guide

Final Project Progress Report: Working Prototype with RAG, Local SLM, Fine-Tuning, or Hybrid AI Solution

Instructor Guide + Colab Notebook Walkthrough

Purpose: This guide supports the Module 7 progress-report milestone. Students demonstrate a working or partially working prototype, explain their pipeline from input to output, report preliminary results, and draft the Methodology and Preliminary Results sections of the final article.

1. Assignment Overview

Students must show that their proposed AI solution can run, produce output, and be evaluated. The prototype may be a RAG pipeline, a local SLM workflow, a guided LoRA/QLoRA experiment, or a hybrid system.

Requirement	What Students Must Show	Evidence Examples
Implementation Progress	At least one working component	Retrieved chunks, generated output, training log, CSV results, screenshot, notebook output
System Description	Pipeline from input to processing to output	Diagram or bullet workflow such as Input -> Chunking -> Retrieval -> LLM -> Evaluation
Local Serving Check	How the model is served locally, or why not	Ollama/LM Studio/llama.cpp/AU lab/Colab explanation
Preliminary Results	At least 5 tests with metrics and observations	Evaluation table, benchmark results, sample outputs
Article Draft Sections	Methodology and Preliminary Results drafts	Markdown or Word sections included in repository or ZIP
AI Usage Disclosure	How AI tools were used and verified	Prompt list, tools used, limitations, verification steps

2. Student Prototype Paths

Path	Best For	Colab-Friendly Version
RAG prototype	Projects that answer questions from documents, policies, FAQs, or manuals	Use 5-10 short documents, TF-IDF or embeddings, top-k retrieval, cited answer draft
Local SLM prototype	Projects that test private/local generation and speed	Use Ollama in Colab or document local runtime on laptop/AU hardware
LoRA/QLoRA fine-tuning prototype	Projects that need consistent format, style, or classification behavior	Use a tiny model or small SLM, a small JSONL dataset, and save an adapter/checkpoint
Hybrid RAG + SLM	Projects that need retrieval plus local generation	Retrieve context first, then generate with local or hosted model and record citations
Comparison study	High-code students comparing approaches	Run baseline vs RAG vs fine-tuning-inspired output using same test cases

3. Recommended In-Class Walkthrough

Time	Instructor Activity	Student Output
0-10 min	Explain that Module 7 is a progress checkpoint, not the final project.	Students identify their project path.
10-20 min	Review required files, Colab folder structure, and evidence expectations.	Students create or download starter notebook.
20-35 min	Run the RAG prototype cells live using fictional IT policy documents.	Students see retrieval, cited context, and answer draft.
35-50 min	Show baseline vs RAG comparison and evaluation table.	Students understand preliminary metrics.
50-65 min	Show optional fine-tuning/LoRA section and explain when to skip it.	High-code students identify whether fine-tuning fits their project.
65-80 min	Generate Methodology and Preliminary Results drafts from notebook outputs.	Students leave with a report skeleton.
80-90 min	Show ZIP export and submission checklist.	Students know exactly what to submit.

4. Colab Notebook Process

Step 1 - Setup: Install Python packages, create project folders, and set project_path.

Step 2 - Create sample data: Generate fictional documents, test questions, and training examples. Students can replace these with their own safe data.

Step 3 - Build a simple RAG pipeline: Chunk documents, vectorize chunks, retrieve top-k passages, and build a citation-friendly RAG prompt.

Step 4 - Generate a prototype answer: Use an extractive answer template for the baseline, then call Ollama for local model generation when the Ollama cells are enabled.

Step 5 - Evaluate: Score retrieval hit rate, citation match, groundedness, format adherence, and response time.

Step 6 - Optional high-code fine-tuning: Create JSONL examples, run LoRA demonstration on a tiny model, save adapter/checkpoint, and compare outputs.

Step 7 - Draft report sections: Automatically write methodology_draft.md and preliminary_results.md.

Step 8 - Export: Save CSVs, charts, markdown drafts, and a ZIP file for LMS/GitHub submission.

5. Local Serving Check Guidance

Every student must address local serving. This does not mean every student must successfully host a large model. They must either show how the model is served locally or justify why they used a Colab-friendly alternative.

If Serving Locally	If Not Serving Locally
Runtime used: Ollama, LM Studio, llama.cpp, AU lab, or similar.	Explain the limitation: hardware, time, memory, GPU availability, or project scope.
Model name and size.	Explain the future plan for local serving or why RAG/evaluation was prioritized first.
Startup command or interface used.	Document the environment actually used, such as Colab, Ollama local API, or the extractive RAG baseline.
Prompt method: CLI, local API, notebook, or GUI.	Still provide preliminary results and metrics.

```
Example local serving evidence:
Runtime: Ollama
Model: qwen2.5:1.5b
Environment: Google Colab VM or AU lab machine
Serving method: http://127.0.0.1:11434/api/generate
Prompt method: Python requests.post() call
Evidence: API health check, generated output, benchmark CSV
```

6. Preliminary Metrics

Metric	Best For	Simple Student Measurement
Retrieval hit rate	RAG	Did the expected source appear in the retrieved top-k chunks?
Citation accuracy	RAG	Does the answer cite the correct document or chunk?
Groundedness	RAG/local generation	Is the answer supported by retrieved context? Score 1-5.
Format adherence	Fine-tuning/structured outputs	Does the answer follow the required format? Score 1-5.
Task accuracy	Classification/summarization	Did the output solve the task correctly? Score 1-5.
Response time	Local SLM	Measure seconds from prompt to response.
Before/after improvement	Fine-tuning/high-code	Compare base vs improved system on same test cases.

7. Required Deliverables

- GitHub repository link or completed ZIP file
- Colab notebook or notebook export
- Progress report with implementation evidence
- Methodology draft
- Preliminary Results draft
- Evaluation table with at least 5 test cases
- Generated outputs and/or retrieved chunks
- AI usage disclosure

Required File	Purpose
README.md	Repository/project overview and student instructions
progress_report.md	Summary of implementation progress, limitations, and next steps
methodology_draft.md	Draft final article Methodology section
preliminary_results.md	Draft final article Results section
ai_usage_disclosure.md	Responsible AI usage explanation
notebooks/module7_project_progress_colab.ipynb	Runnable or reviewable Colab notebook
results/evaluation_scores.csv	At least 5 test cases with metrics
results/generated_outputs.csv	Model or prototype outputs

8. High-Code Track Guidance

High-code students should add modularity and automated evaluation. The provided notebook includes sections for RAG, local generation through Ollama, and a small LoRA fine-tuning demonstration. Hosted API-key cells have been removed.

High-Code Feature	What It Adds
Automated evaluation loop	Runs all test cases and saves scores to CSV
Reusable retrieval function	Makes RAG easier to test and improve
Reusable generation function	Allows baseline/RAG/local/API comparisons
Benchmark logging	Tracks response time and output length
Charts	Shows preliminary performance visually
LoRA adapter demo	Shows a fine-tuning workflow and saves a checkpoint/adapter
RAG vs fine-tuning decision table	Connects technical evidence to design justification

9. Common Colab Troubleshooting

Problem	Cause	Fix
Notebook runs slowly	Colab CPU or small VM resources	Use fewer documents, fewer training steps, or skip optional fine-tuning cells.
Ollama connection refused	Ollama server is not running	Run the server cell and verify /api/version before calling /api/generate.
Model download fails	Network, model name, or memory issue	Use the smaller Ollama fallback model or continue with the extractive RAG baseline while documenting the local-serving limitation.
CUDA out of memory	Model/training too large for GPU	Use the tiny LoRA demo model, reduce batch size, or skip training.
Poor model quality	Tiny model or too few examples	Explain limitation honestly; grade is based on method and evaluation.
No local serving	Hardware limitations	Justify and provide a future local-serving plan.

10. Student Writing Templates

Methodology Template

```
## Methodology
This project uses [RAG / local SLM / fine-tuning / hybrid approach] to solve [problem].
The input data consists of [documents / tickets / prompts / policies]. The data was prepared by
[cleaning, formatting, chunking, converting to JSONL, etc.].
The system pipeline is: Input -> Processing -> Model or Retrieval -> Output -> Evaluation.
For evaluation, I used [metrics]. The system was tested on [number] examples to measure [expected
outcome].
```

Preliminary Results Template

```
## Preliminary Results
The current prototype was tested on [number] examples.
The early results show that the system performs well on [strength]. However, it still struggles with
[limitation].
The baseline method was [baseline]. Compared with the baseline, the prototype [improved / partially
improved / did not improve] in [metric].
The most important issue to fix before the final submission is [issue].
```

AI Usage Disclosure Template

```
# AI Usage Disclosure
AI tools used: [ChatGPT / Claude / GitHub Copilot / Hugging Face / Ollama / Other]
How I used AI: [brainstorming, code support, debugging, documentation, evaluation]
Prompts used: [list important prompts]
What I verified myself: [outputs, metrics, citations, code behavior]
Failures or limitations: [weak outputs, errors, hallucinations, formatting failures]
Academic integrity statement: I confirm that AI was used as a learning and support tool, not as a
replacement for my own work.
```

11. Suggested Grading Rubric

Criteria	Points
Implementation progress is clearly demonstrated with evidence	20
System pipeline is clearly described from input to output	15
Local serving check is completed or justified	10
Preliminary results include metrics, observations, and at least 5 tests	20
Methodology draft is clear and technically appropriate	15
Results draft summarizes early findings and limitations	10
Colab notebook, GitHub, or ZIP package is organized and runnable	5
AI usage disclosure is complete	5
Total	100

12. Recommended Student Resources

Resource	Use
Hugging Face PEFT: https://huggingface.co/docs/peft/en/index	LoRA and parameter-efficient fine-tuning reference
Hugging Face bitsandbytes: https://huggingface.co/docs/bitsandbytes/main/en/index	Quantization and QLoRA-related memory savings
Hugging Face TRL SFTTrainer: https://huggingface.co/docs/trl/en/sft_trainer	Supervised fine-tuning workflow reference
Ollama Linux docs: https://docs.ollama.com/linux	Local model serving setup and manual install commands
Google Colab: https://colab.research.google.com/	Notebook runtime for prototype and high-code track

13. Instructor Final Checklist

- Students understand that this is a progress milestone, not final completion.
- Students can explain their pipeline in one sentence.
- Students have at least 5 test cases.
- Students know how to record preliminary metrics.
- Students know what to submit as GitHub or ZIP.
- Students know how to justify local serving if it cannot be completed.